



Цифровые технологии управления
транспортными процессами

Сервис-ориентированное
программирование



RPC



Удаленный вызов процедур

- Удалённый вызов процедур (от англ. Remote Procedure Call, RPC) — класс технологий, позволяющих компьютерным программам вызывать функции или процедуры в другом адресном пространстве (как правило, на удалённых компьютерах).
- Средства удаленного вызова процедур предназначены для облегчения организации распределенных вычислений.
- Обычно реализация RPC технологии включает в себя два компонента: сетевой протокол для обмена в режиме клиент-сервер и язык сериализации объектов (или структур, для необъектных RPC).
- Различные реализации RPC имеют очень отличающуюся друг от друга архитектуру и разнятся в своих возможностях: одни реализуют архитектуру SOA, другие CORBA или DCOM. На транспортном уровне RPC используют в основном протоколы TCP и UDP.



Существует множество технологий, обеспечивающих RPC:

- DCE/RPC — Distributed Computing Environment / Remote Procedure Calls (бинарный протокол на базе различных транспортных протоколов, в том числе TCP/IP и Named Pipes из протокола SMB/CIFS)
- DCOM — Distributed Component Object Model известный как MSRPC Microsoft Remote Procedure Call или «Network OLE» (объектно-ориентированное расширение DCE RPC, позволяющее передавать ссылки на объекты и вызывать методы объектов через таковые ссылки)
- JSON-RPC— JavaScript Object Notation Remote Procedure Calls (текстовый протокол на базе HTTP) см. спецификацию: RFC-4627
- NET Remoting (бинарный протокол на базе TCP, UDP, HTTP)
- Java RMI — Java Remote Method Invocation — см. спецификацию: <http://java.sun.com/j2se/1.5.0/docs/guide/rmi/index.html>
- SOAP — Simple Object Access Protocol (текстовый протокол на базе HTTP) см. спецификацию: RFC-4227
- XML RPC

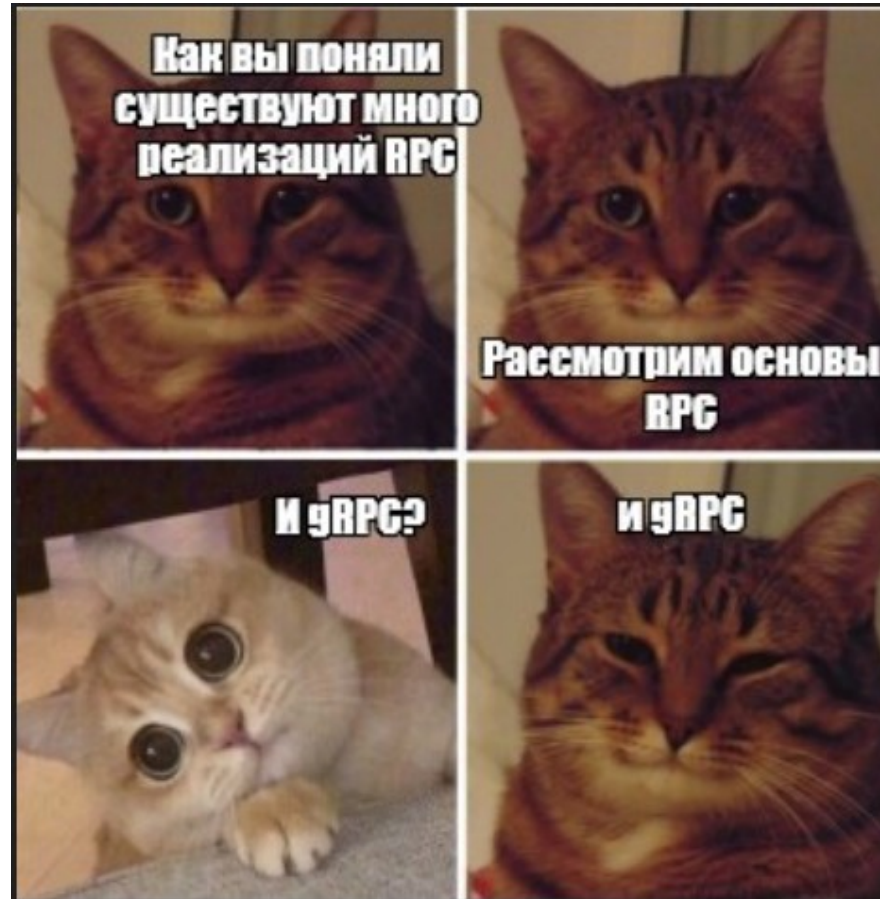


Существует множество технологий, обеспечивающих RPC:

RPC system	Transport	Language support	Notes
Sun/ONC RPC	Custom TCP or UDP	Various (Mainly C)	Early standard for RPC. Used in NFS (Network File System). Custom "Rpcgen" IDL.
CORBA	Custom TCP	Various	Early standard for RPC and distributed objects. Custom IDL.
DCOM	Custom TCP	Various (Windows Only)	Distributed objects. Proprietary Microsoft serialization. Supports distributed garbage collection. Custom IDL.
Java RMI	Custom TCP	Java	Distributed objects. Uses Java serialization.
XML-RPC, SOAP	HTTP 1.1	Various	Specifies the communication protocol, not the client and server programming models. Uses XML for message encoding.
JSON-RPC	HTTP 1.1	Various	Specifies the communication protocol, not the client and server programming models. Uses JSON for message encoding. Technically a subset of REST.
Apache Thrift	Custom TCP	Various	Custom IDL that is similar to protobuf. No streaming.
Apache Avro	Custom TCP or HTTP 1.1	Various	Code generation not required. Data includes schema. IDL is formatted as JSON.
Go net/rpc	HTTP 1.1	Go	Uses Go's GOB encoding.
Cap'n Proto	Custom TCP	Various (Mainly C++)	Distributed objects. Promise pipelining. Strives for zero-overhead serialization. Custom IDL.
Twirp	HTTP 1.1	Various	No streaming. Can use JSON as encoding. Uses Protobuf as IDL.



Время для мема есть всегда





Характерными чертами вызова локальных процедур являются:

- асимметричность, то есть одна из взаимодействующих сторон является инициатором;
- синхронность, то есть выполнение вызывающей процедуры приостанавливается с момента выдачи запроса и возобновляется только после возврата из вызываемой процедуры.



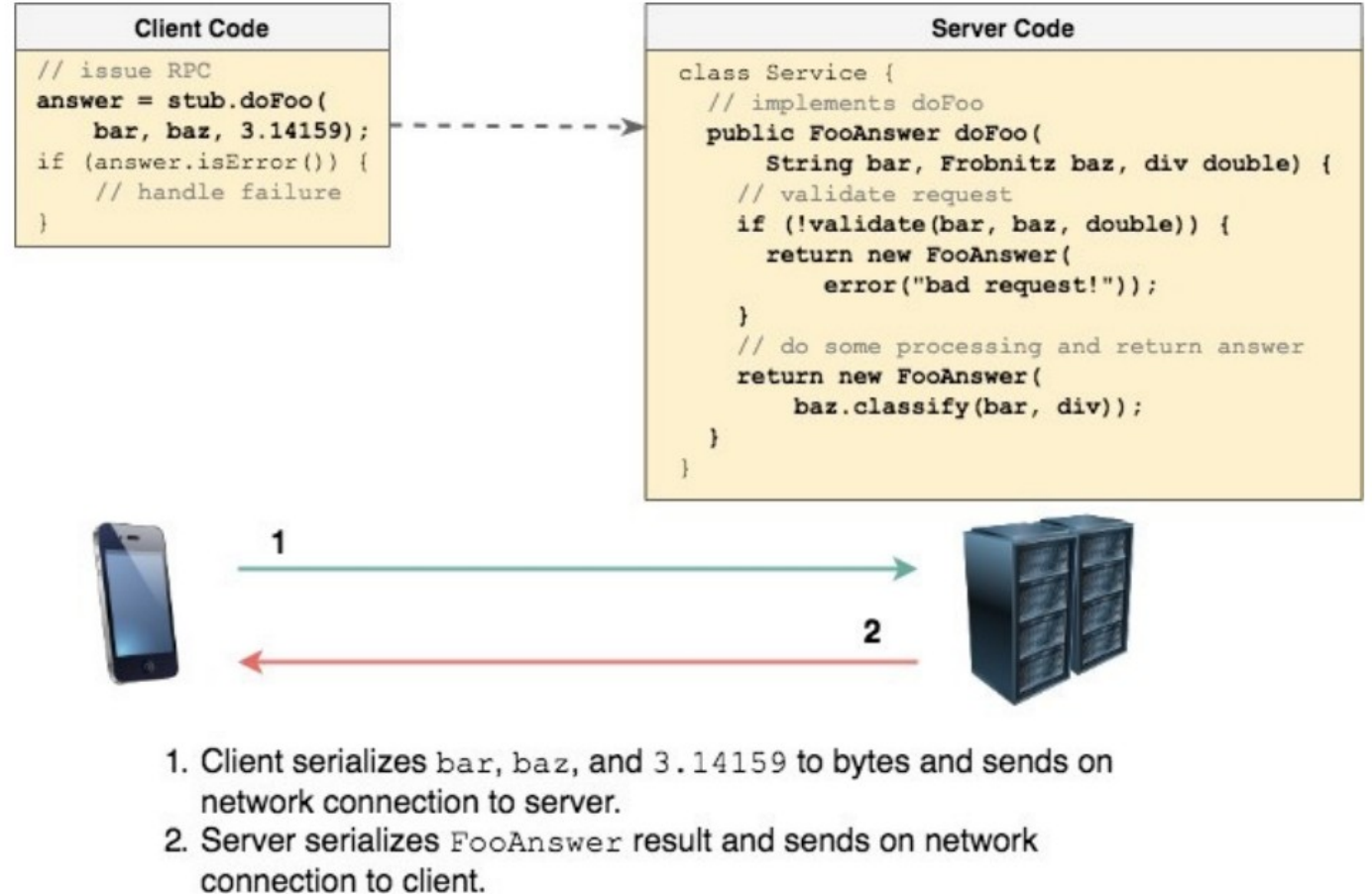
Сложности реализации

- Реализация удаленных вызовов существенно сложнее реализации вызовов локальных процедур.
- Поскольку вызывающая и вызываемая процедуры выполняются на разных машинах, они имеют разные адресные пространства, и это создает проблемы при передаче параметров и результатов, особенно если машины не идентичны.
- Так как RPC не может рассчитывать на разделяемую память, это означает, что параметры RPC не должны содержать указателей на ячейки не стековой памяти и что значения параметров должны копироваться с одного компьютера на другой.
- Следующее отличие RPC от локального вызова заключается в том, что он обязательно использует нижележащую систему связи, однако это не должно быть явно видно ни в определении процедур, ни в самих процедурах.



Идея

Идея, положенная в основу RPC, состоит в том, чтобы вызов удаленной процедуры выглядел по возможности так же, как вызов локальной процедуры. Другими словами - сделать RPC прозрачным: вызывающей процедуре не нужно знать, что вызываемая процедура находится на другой машине, и наоборот.





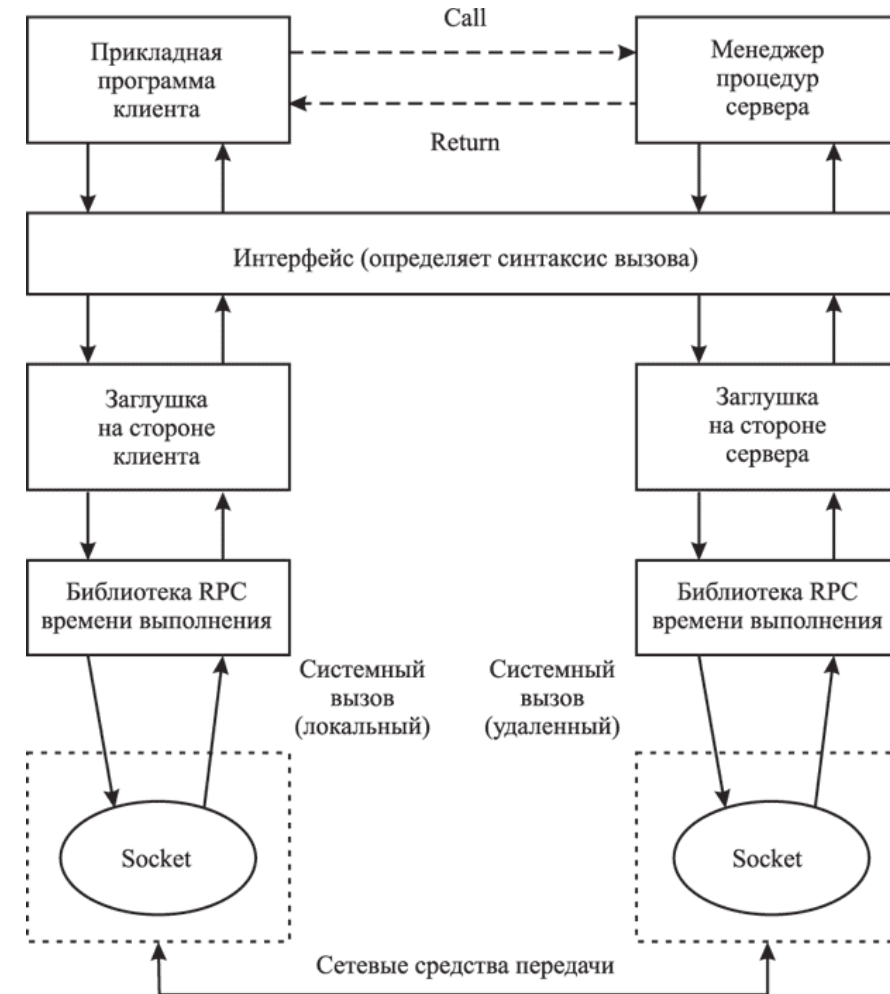
Удаленный вызов процедур

- RPC достигает прозрачности следующим путем. Когда вызываемая процедура действительно является удаленной, вместо локальной процедуры используется другая ее версия, называемая клиентской заглушкой (stub). Подобно локальной процедуре, заглушка вызывается с использованием того же списка параметров, только в отличие от локальной процедуры она отправляет вызывающее сообщение на сервер.
- Вызывающее сообщение содержит параметры процедуры для процесса на сервере. Далее клиентский процесс ожидает от сервера ответного сообщения. Процесс на серверной стороне ожидает поступления вызывающего сообщения, дождавшись которого, извлекает параметры процедуры, выполняет необходимые вычисления и отправляет ответное сообщение клиенту.



Удаленный вызов процедур

Далее процесс на серверной стороне ждет следующего вызывающего сообщения. Процесс на клиентской стороне получает ответное сообщение, извлекает из него результаты вычислений и продолжает выполнение.





Удаленный вызов процедур

Интерфейс RPC не зависит от транспортных протоколов. В зависимости от ситуации, на транспортном уровне могут быть использованы или протокол TCP, или протокол UDP. В общем случае, взаимодействующие процессы пользуются механизмом сокетных соединений



Цифровые технологии управления
транспортными процессами

Сервис-ориентированное
программирование



gRPC



Проблематика

- Много сервисов – это много API
 - Приходится создавать множество разных интеграций с разными API
 - Приходится каждый раз писать обвязку кода
 - Для одного клиентского запроса приходится много раз ходить по сети



Проблематика

- Что привело к росту интереса к протоколам, которые в свою очередь:
 - Имеют Interface Definition Language из коробки с проверкой типов и автоматической валидацией
 - Избавляющие разработчиков от необходимости писать повторяющийся код и думать про транспортный уровень
 - Более производительный, как по размеру передаваемых данных, так и по использованию сетевых соединений
- В результате возник ряд RPC-протоколов Thrift, gRPC, Avro ...



RPC все более и более популярен

	Thrift (fbthrift)	
 badoo	TCP + Proto2	—————→ gRPC
	HTTP + JSON	—————→ Thrift
 Dropbox	HTTP + Proto, Thrift	—————→ gRPC
	HTTP + JSON, Thrift, gRPC	————→ Twirp
	UDP + Custom Binary	—————→ UDP + Proto3
 Yandex	HTTP + JSON, XML	—————→ gRPC

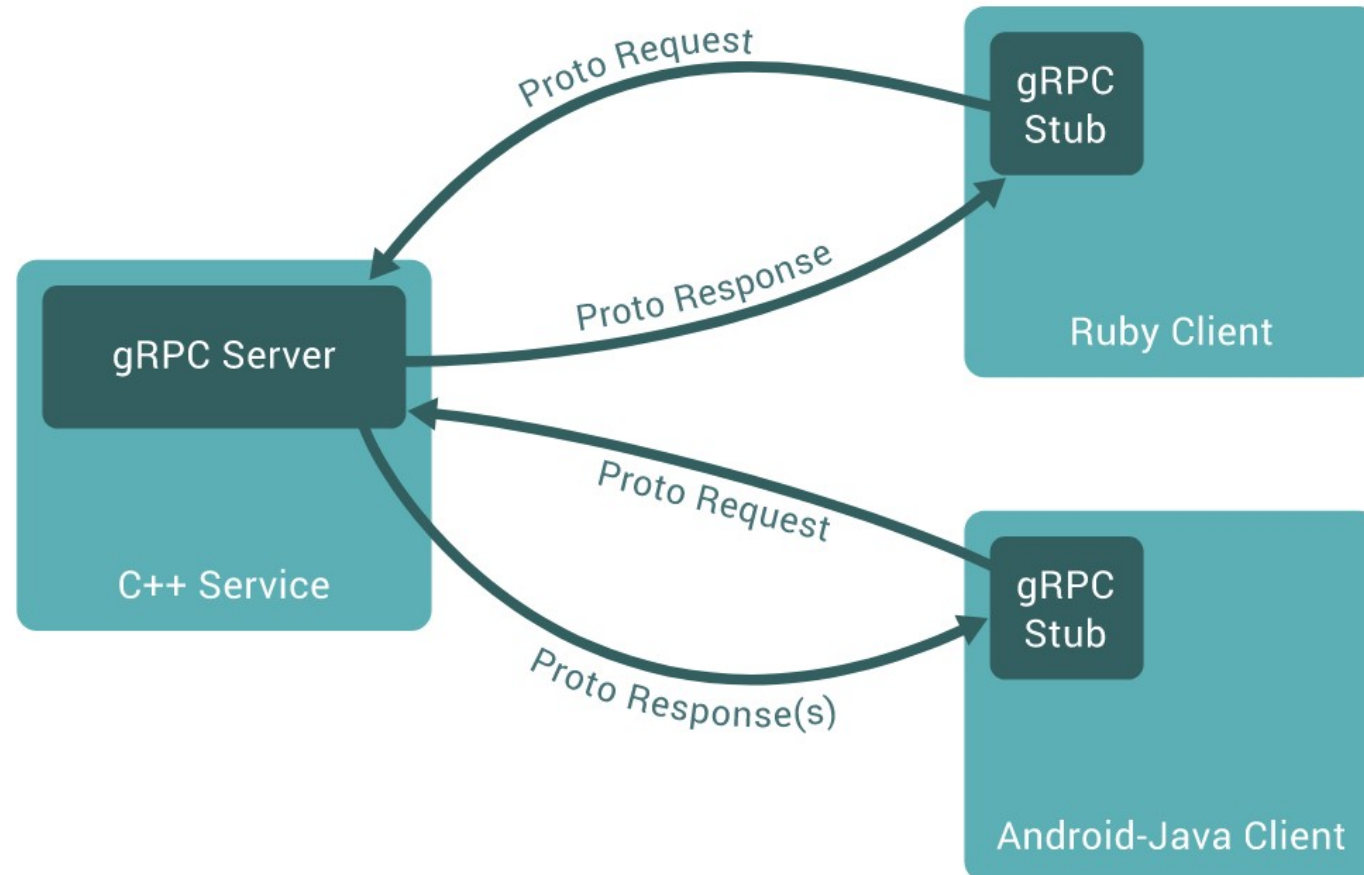


gRPC

- gRPC — кросс-платформенный протокол удаленного вызова процедур от Google.
- gRPC простой в использовании, отлично подходит для создания распределенных систем (микросервисов) и API. Имеет встроенную поддержку для балансировки нагрузки, трассировки, аутентификации и проверки жизнеспособности сервисов. Есть возможность создавать клиентские библиотеки для работы с бэкендом на 10 языках. Высокая производительность достигается за счет использования протокола HTTP/2 и Protocol Buffers.
- Используется IDL и формат данных на основе protobuf
- Использует кодогенерацию для stub-ов



gRPC





Чем gRPC лучше REST

- REST — не Contract First: сложно управлять контрактами и целостностью API. По сути целостность контрактов нельзя контролировать. Можно прикрутить Swagger, инструмент для генерации документации по коду. То есть у вас уже есть код и генерируется документация. А хотелось бы по понятным причинам наоборот: чтобы код генерировался по документации.
- Много бойлерплейт-кода: создать HTTP-клиент, HTTP client factory, договориться о сериализации (JSON, XML или другие варианты), учесть различные форматы дат, чисел и так далее, кейсинг. Соответственно, много возможностей для возникновения ошибок — нужно писать DTO-классы специально для транспорта.
- У REST не очень хороший перформанс в сравнении с gRPC.
- Использовать REST не очень оптимально с точки зрения сети, потому что будет летать JSON или XML. Правда, этот момент можно улучшить: использовать msgPack или похожие средства оптимизации транспорта.



Что предлагает gRPC

- Contract First. Сначала договариваемся о том, что и куда пересылаем, описываем это строго в protobuf-файле, кладем в репозиторий, и, по сути, с этого момента он зафиксирован. При желании от этого решения можно отказаться и пользоваться Code First. Но это применимо, только если у вас весь бэкенд на .NET: можно создать те самые DTO, повесить на них атрибуты привычным вам способом, как с тем же JSON или XML.
- Строгая типизация.
- Автогенерация клиентского и серверного кода. Описываете контракт, используя штатный инструмент во время билда, и у вас создается либо клиент, либо сервер, либо и то и другое — в зависимости от настройки.
- Это дает сразу несколько преимуществ:
 - При кодогенерации не нужно писать код самому и поддерживать его.
 - Нельзя сбилдить проект, не поменяв клиент и сервер, если поменялся контракт.



Что предлагает gRPC

- Встроена сериализация из коробки.
 - Она достаточно эффективна и с точки зрения перформанса, и с точки зрения транспорта: пейлоад, который летает по сети, существенно компактней обычного JSON.
- У gRPC достаточно лаконично сделана обработка ошибок. Какие-то вещи она маскирует и упрощает, но для практического применения ее вполне достаточно.
- gRPC предлагает удобный стриминг, то есть возможность создать стрим в любую сторону: с сервера на клиент, с клиента на сервер и даже двунаправленный стрим — дуплекс.



Protocol Buffers (protobuf)

- Protobuf формат сериализации используемый по умолчанию для передачи данных между клиентом и сервером. Используя строгую типизацию полей и бинарный формат для передачи структурированных данных потребляет меньше ресурсов. Время выполнения процесса сериализации/десериализации значительно меньше как и размер сообщений в отличии от JSON/XML.
- Для написания protobuf файлов используют язык описания интерфейсов (IDL, формат файлов - .proto). Например, чтобы описать структуру данных сообщения, нужно добавить message, имя структуры, а внутри тип, название и номер поля. Номера полей очень важны для обратной совместимости, поэтому не стоит менять их последовательность при добавлении или удалении полей.



IDL

(<https://developers.google.com/protocol-buffers/docs/proto3>)

```
message Person {  
  string name = 1;  
  int32 id = 2;  // Unique ID number for this person.  
  string email = 3;  
  
  enum PhoneType {  
    MOBILE = 0;  
    HOME = 1;  
    WORK = 2;  
  }  
  
  message PhoneNumber {  
    string number = 1;  
    PhoneType type = 2;  
  }  
  
  repeated PhoneNumber phones = 4;  
  
  google.protobuf.Timestamp last_updated = 5;  
}
```



Кодогенерация кода

- Генерируются и клиент, и сервер, и сериализация, и десериализация, и DTO, то есть весь этот слой будет у вас готов — его не нужно писать.
- Поддерживается несколько популярных платформ: C#, Go, Java, Python, Kotlin, PHP, Ruby. Поддерживается не только бэкенд — стандартные стеки, но и клиентские мобильные платформы.

```
protoc -I=$SRC_DIR --csharp_out=$DST_DIR $SRC_DIR/person.proto
```



Типы RPC

- Унарный (Unary RPC). Синхронный запрос клиента, который блокируются пока не будет получен ответ от сервера.
- Серверный стрим (Server streaming RPC), при подключении клиента сервер открывает стрим и начинает отправлять сообщения.
- Клиентский стрим (Client streaming RPC). То же самое, что и серверный, только клиент начинает стримить сообщения на сервер.
- Двухнаправленный стрим (Bidirectional streaming). Клиент инициализирует соединение, создаются два стрима. Сервер может отправить изначальные данные при подключении или отвечать на каждый запрос клиента по типу “пинг-понга”.



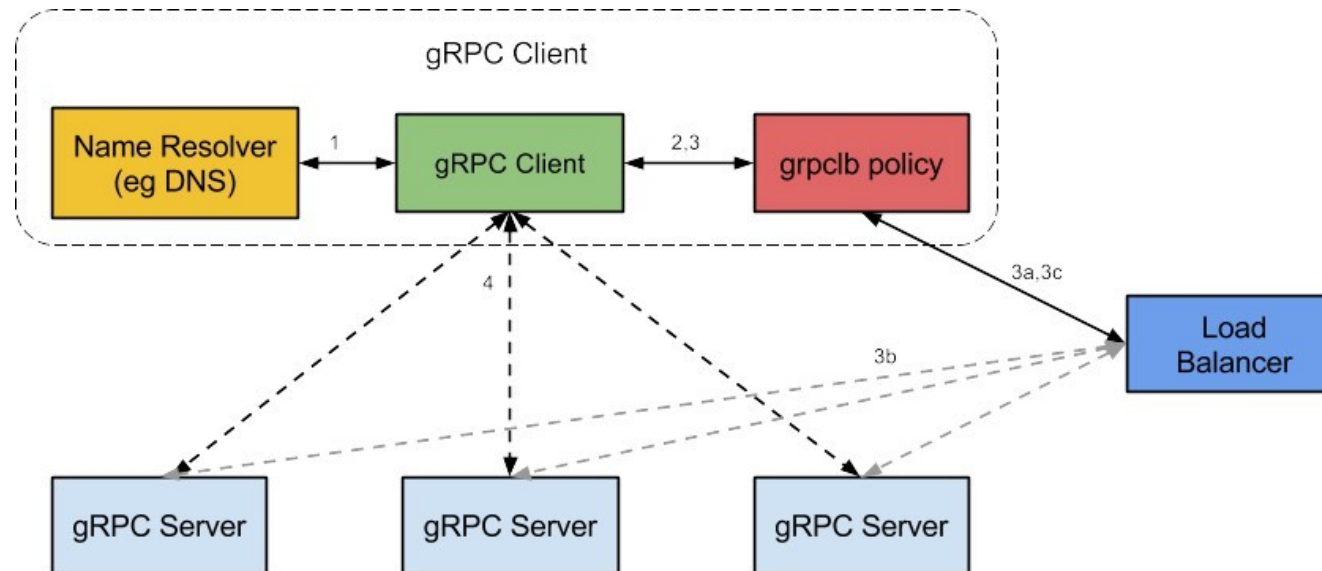
Перехватчики (Interceptors)

Выступают этакой заменой middleware, служат для перехвата вызовов и встраивания необходимых условий. Отлично подходят для логирования запросов, сбора метрик, аутентификации и т.д.



Балансировка нагрузки (Load Balancing)

Выполняется на стороне клиента. Клиент использует простой «round-robin» алгоритм для передачи запросов по списку полученному от LB сервера. При желании на стороне LB сервера можно организовать более сложный алгоритм выдачи списка бэкенд сервисов клиенту используя LB политики.





Критика

- Сложный runtime.
- В случае багов в runtime вообще непонятно, что делать.
- Есть проблемы с load balancing
- Streaming практически никому на самом деле не нужен, а его наличие значительно усложняет рантайм и создает баги.
- Бинарный протокол не сильно помогает дебагу, а инструменты для работы с gRPC сложны и очень плохо развиты на данный момент не поддерживает http1.1, что усложняет использование с балансерами, которые не все нормально умеют поддерживать http/2.
- Чаще всего время на создание соединения и передачу данных не является бутылочным горлышком.